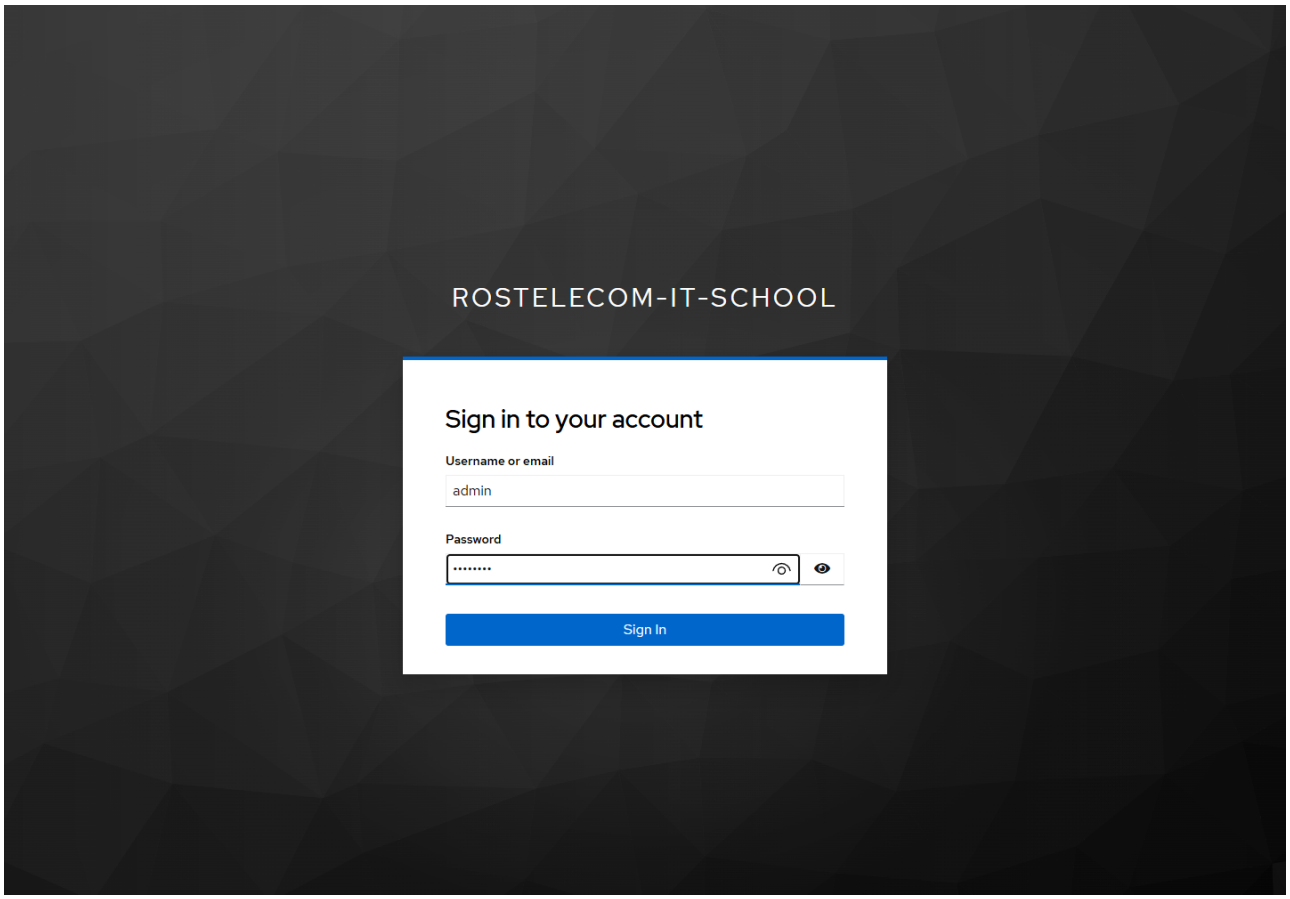
Руководство пользователя

Вход

Откройте http://localhost:3000. Браузер перенаправит вас в Keycloak. В учебном стенде доступны admin/admin123, manager/manager123 и user/user123. Выйти можно кнопкой в правом верхнем углу. Пароль не передаётся в CRM и не хранится в браузерном localStorage.

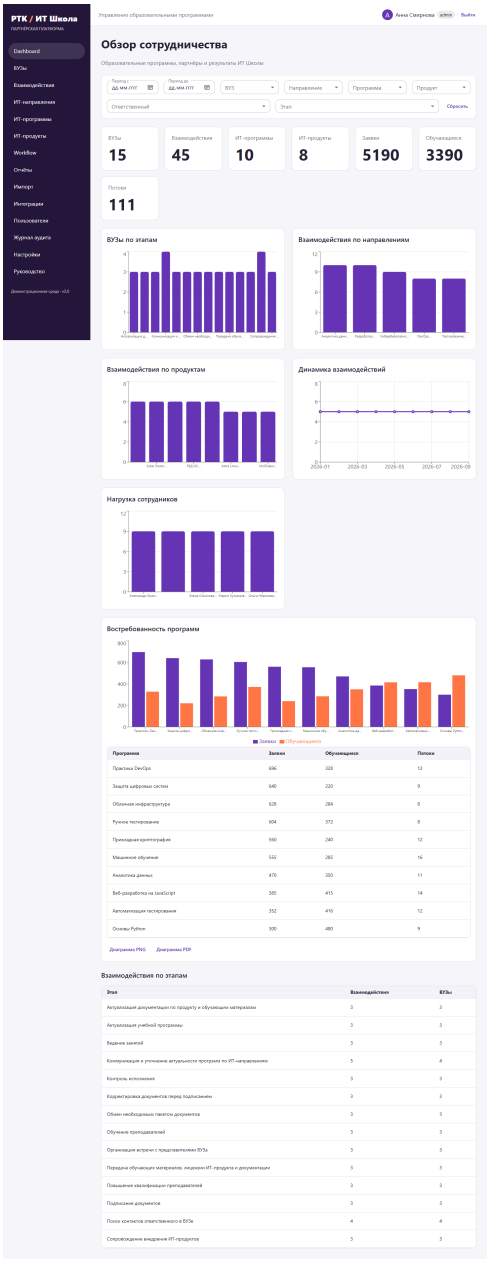


Вход через Keycloak

Обзор и фильтры

Dashboard показывает показатели только по доступным взаимодействиям: ВУЗы, программы, продукты, заявки, обучающиеся и потоки. Период относится к дате создания взаимодействия; обе границы включены. Фильтры объединяются условием AND. «Сбросить» возвращает всю доступную выборку.

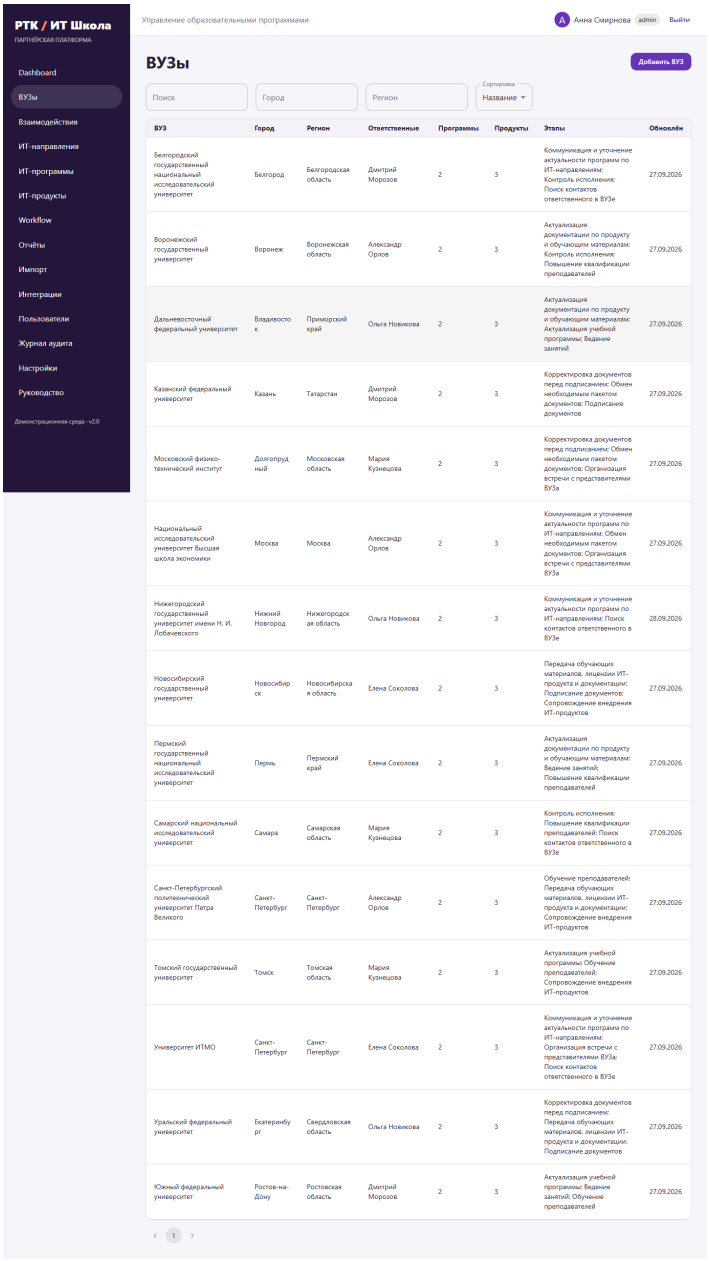
Пять графиков показывают ВУЗы по этапам, взаимодействия по направлениям, продуктам, месяцам и сотрудникам. Один ВУЗ может находиться на нескольких этапах по разным программам. Поэтому сумма ВУЗов по этапам не обязана совпадать с числом уникальных ВУЗов. Все показатели берутся из БД. Рейтинг программ упорядочен по сумме заявок; дополнительно видны обучающиеся и потоки. Диаграмма рейтинга скачивается в PNG или PDF.



Dashboard

ВУЗы и контакты

Раздел «ВУЗы» поддерживает поиск, фильтры города и региона, сортировку и постраничный вывод. Нажмите строку для открытия карточки. В карточке находятся контакты, ответственные, программы, продукты, договоры, лицензии и взаимодействия. Контакт можно добавить или изменить нажатием на строку. Администратор создаёт и редактирует ВУЗы. Руководитель назначает, меняет и снимает ответственных сотрудников своей команды.



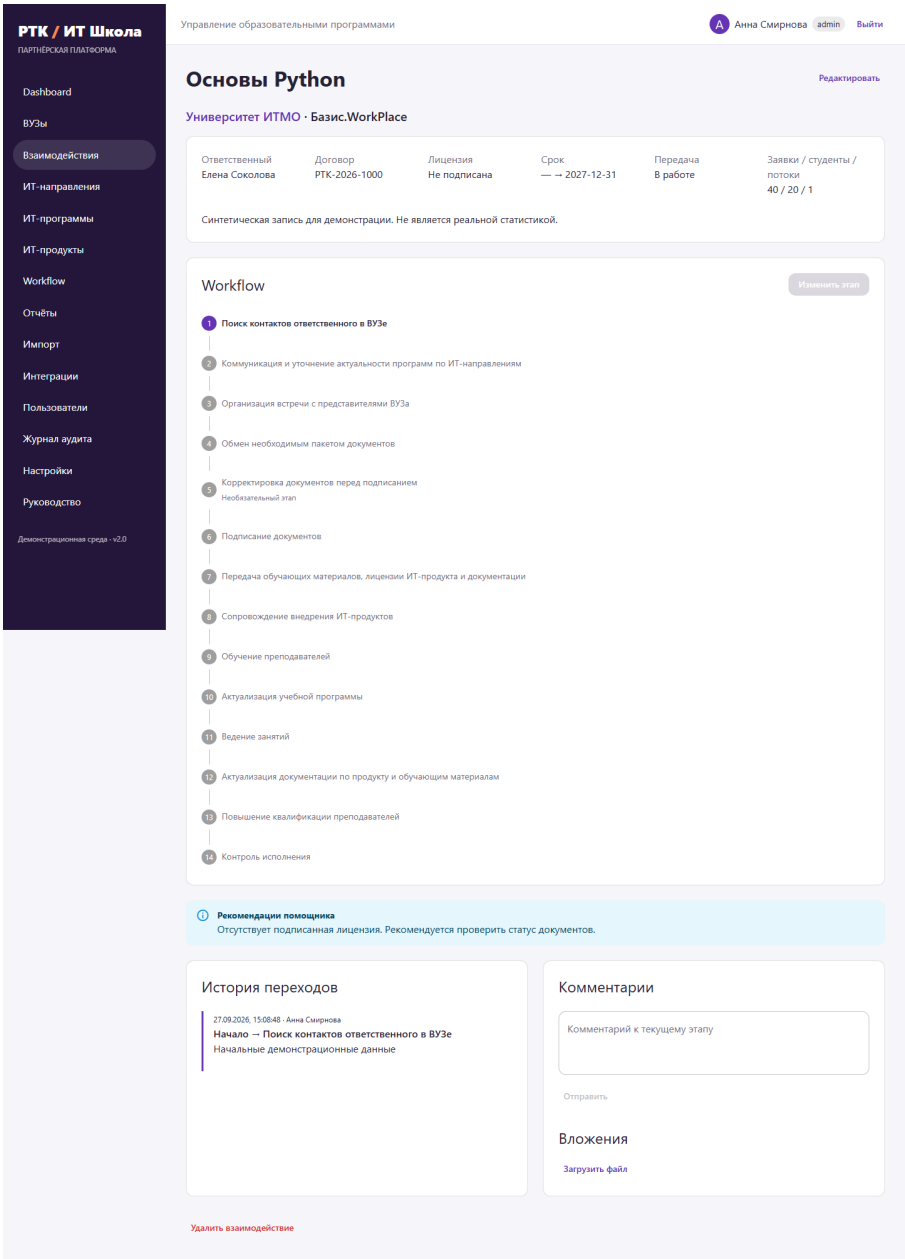
ВУЗы

Взаимодействие и workflow

Создайте взаимодействие, выбрав ВУЗ, программу, продукт и workflow. Оно начинается на первом активном этапе. В существующей записи можно изменить договор, даты лицензии, показатели обучения, статус передачи и примечание. ВУЗ и workflow существующей записи неизменяемы, чтобы сохранить целостность истории.

Вертикальный stepper показывает весь путь. «Изменить этап» открывает список разрешённых переходов. Пользователь идёт вперёд; необязательный этап может пропускаться. Руководитель и администратор могут корректировать путь в пределах настроенных прав. Отключённые этапы сохраняются в истории, но недоступны для новых переходов. Каждый переход содержит старый и новый этап, автора, время и комментарий.

В форме перехода можно выбрать файл. Если этап изменился, но файл не загрузился, интерфейс сообщает об этом отдельно: повторите загрузку в разделе вложений. Комментарии и файлы привязаны к этапу. При одновременной правке одной записи сервер возвращает конфликт версии: перечитайте карточку перед повторной операцией.



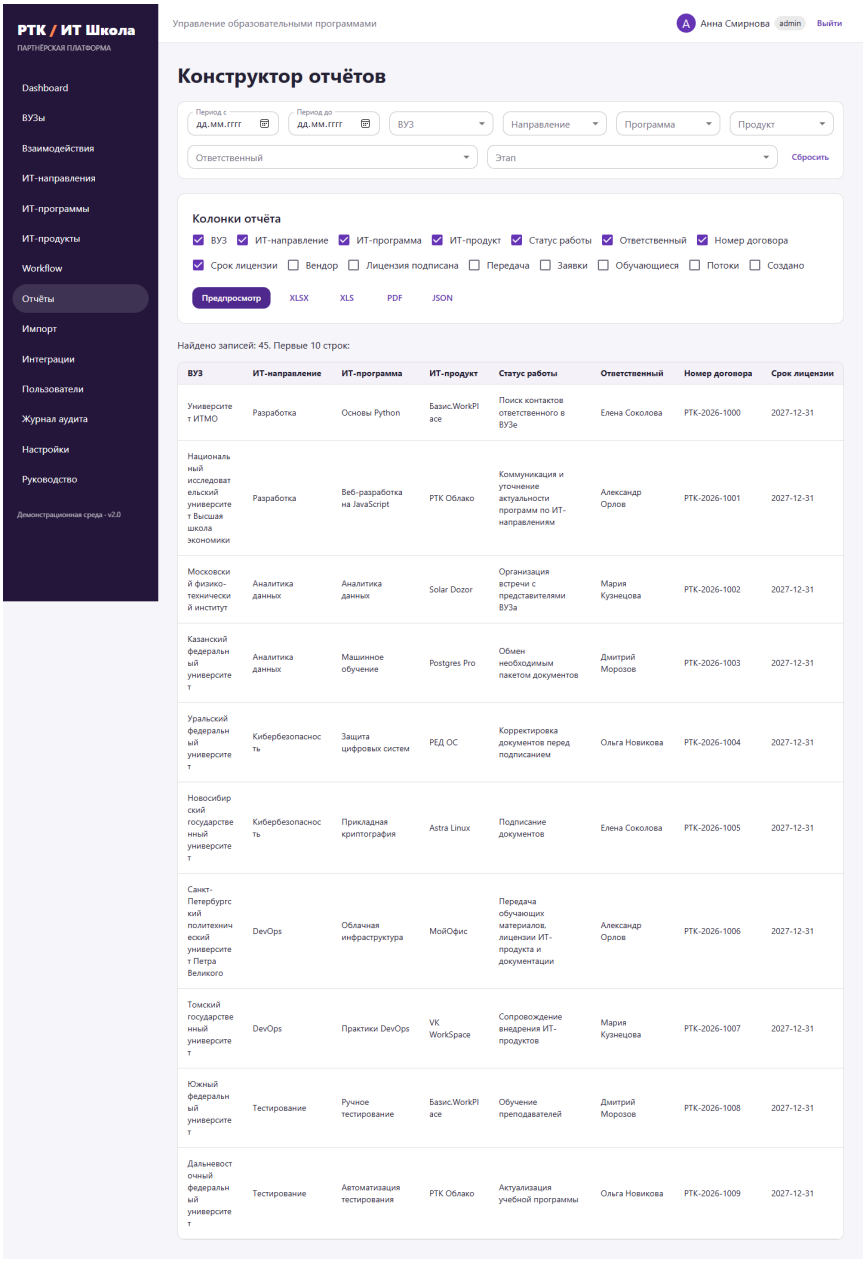
Карточка взаимодействия

Документы

Разрешены PNG, JPG/JPEG, PDF, ZIP, GZIP/GZ, RAR, DOC/DOCX, XLS/XLSX. По умолчанию максимум 20 МБ. Проверяются расширение, MIME и сигнатура. Файлы скачиваются после повторной проверки доступа. Имя в хранилище случайное. Архивы не распаковываются в пользовательские каталоги.

Отчёты

Откройте «Отчёты», установите фильтры, отметьте колонки и нажмите «Предпросмотр». Показываются первые 10 строк и общее количество. Скачать можно XLSX, XLS, PDF или JSON. PDF содержит заголовок, период, дату формирования, фильтры и таблицу. Лимит одного отчёта — 50 000 строк; при превышении сузьте выборку.

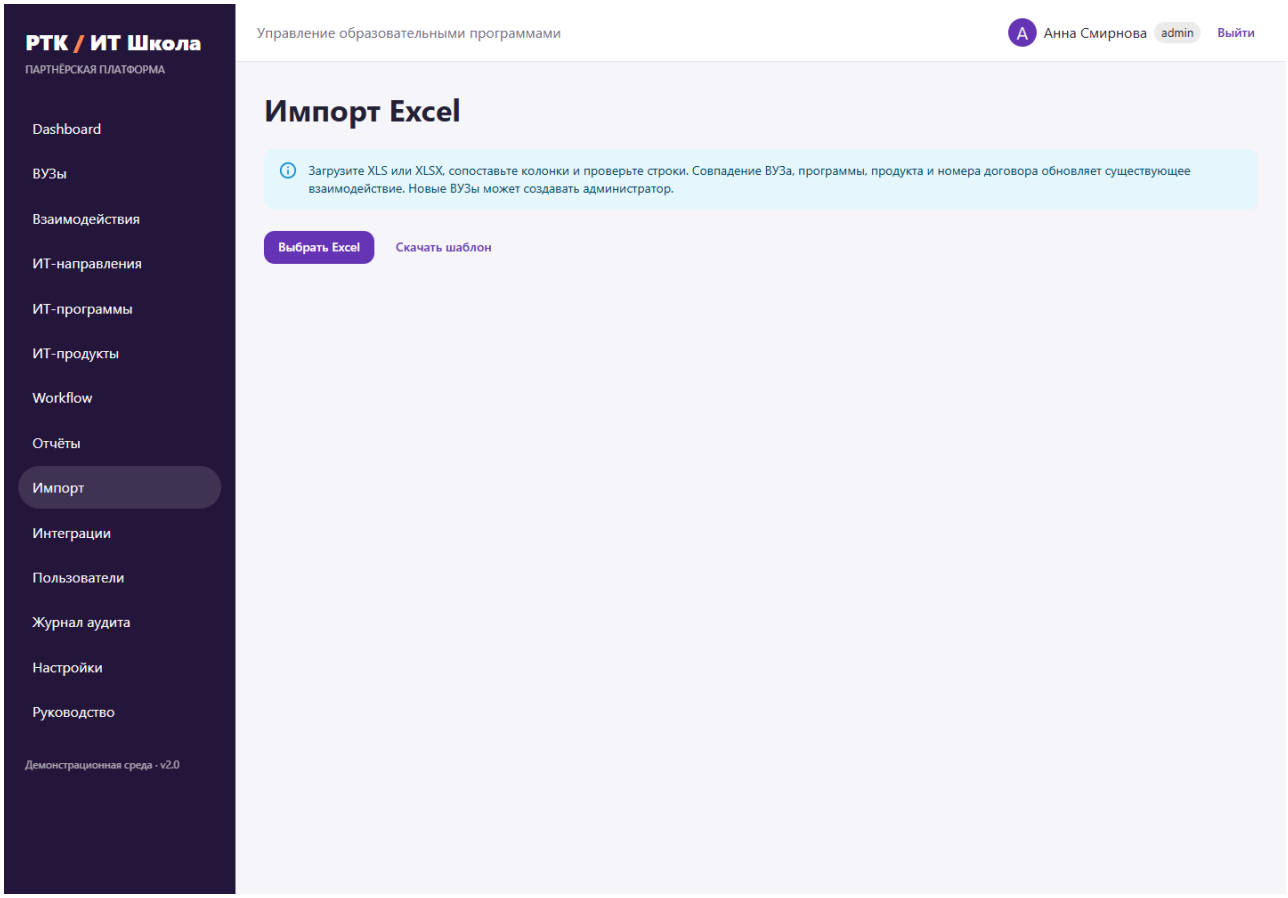


Конструктор отчёта

Импорт

Руководителю и администратору доступен раздел «Импорт». Скачайте шаблон, заполните его, загрузите XLS/XLSX. Сопоставьте колонки: обязательны ВУЗ и ПО, программа задаётся колонкой либо выбором по умолчанию. Выберите workflow для новых записей, проверьте preview и подтвердите импорт. В результате видны созданные, обновлённые, пропущенные строки и конкретные ошибки. Новые ВУЗы создаёт только администратор. ФИО менеджера должно совпадать с существующим активным сотрудником; импорт не создаёт ему учётную запись Keycloak.

Поля лицензии принимают «Да/Нет», дату или год окончания. Даты рекомендуется задавать как YYYY-MM-DD, год превращается в 31 декабря. Совпадение ВУЗа, программы, продукта и непустого номера договора обновляет запись. Без номера договора строки создаются как новые. Существующие поля, не переданные в файле, сохраняются. Каждая строка обрабатывается независимо: ошибка не отменяет удачные строки. Максимум 10 000 строк.



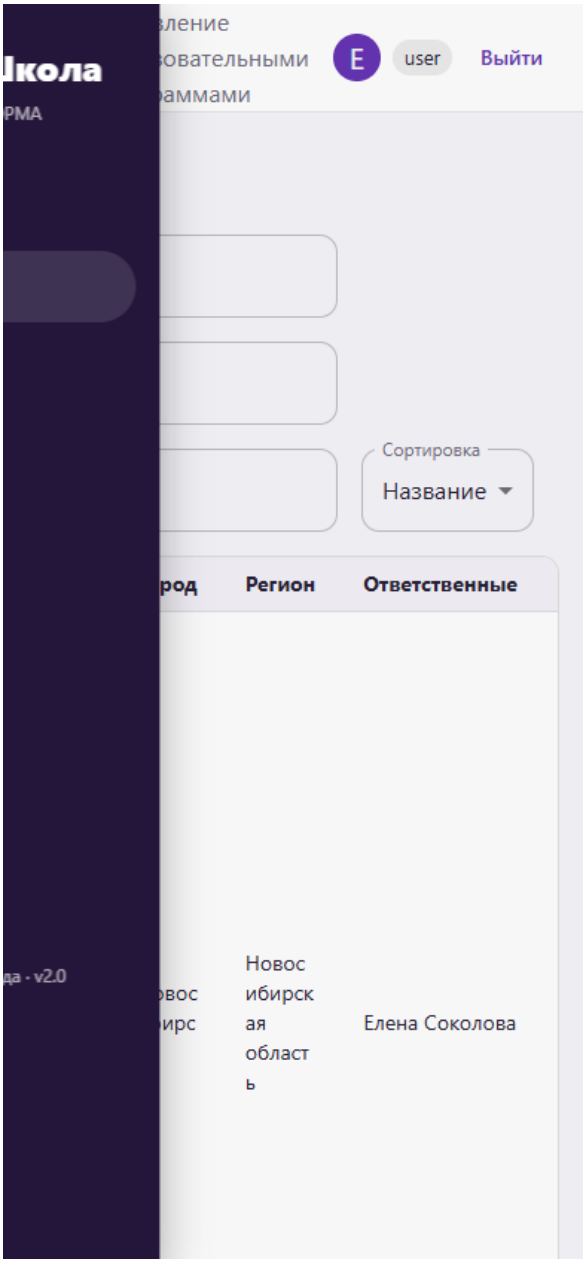
Импорт и маппинг

Интеграции и помощник

Кнопки LMS и Website запускают синхронизацию с выбранным адаптером. В деморежиме используются стабильные тестовые записи. Повторная синхронизация опирается на source + external_id. Права текущего пользователя сохраняются, поэтому импорт недоступного ВУЗа попадёт в ошибки. Помощник в карточке предлагает проверить лицензию, договор, ответственного и отсутствие изменений более 14 дней. Он работает локально по правилам и ничего не отправляет внешней LLM.

Ошибки и мобильная версия

401 — сессия недействительна; войдите повторно. 403 — недостаточно прав. 404 — запись отсутствует или недоступна. 409 — конфликт версии или связанных данных. 413 — файл слишком большой. 415 — неподдерживаемый файл. 422 — некорректные поля. При проблеме подключения повторите действие после восстановления связи. На телефоне меню открывается кнопкой слева сверху; широкие таблицы прокручиваются горизонтально.



Мобильное меню

Руководство администратора

Первый запуск

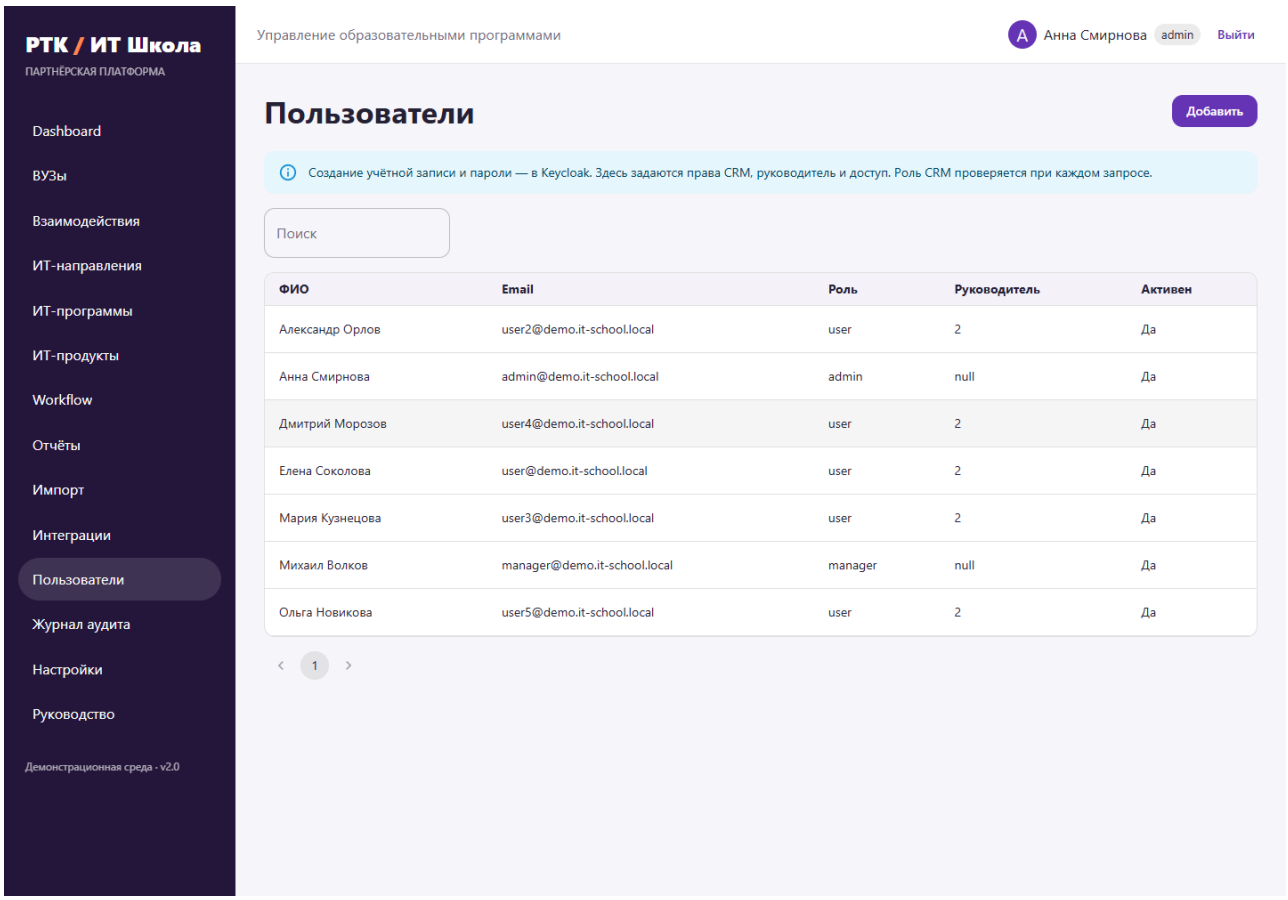
Установите Docker Engine и Compose v2 либо Docker Desktop с Linux containers. Из корня выполните docker compose up --build -d. Сервисы запускаются в порядке PostgreSQL → Keycloak и Redis → миграция/seed/backend → frontend. Первый запуск требует доступа к реестрам образов и пакетам. Логи: docker compose logs -f backend keycloak. Готовность: docker compose ps и http://localhost:8000/health.

Пользователи и роли

Realm: rostelecom-it-school. Clients: crm-frontend (public, Authorization Code + PKCE S256), crm-backend (audience). Демопрофили содержат обязательные имя, фамилию и email. Учётная запись bootstrap-admin служит для консоли Keycloak; это отдельный пользователь от admin CRM.

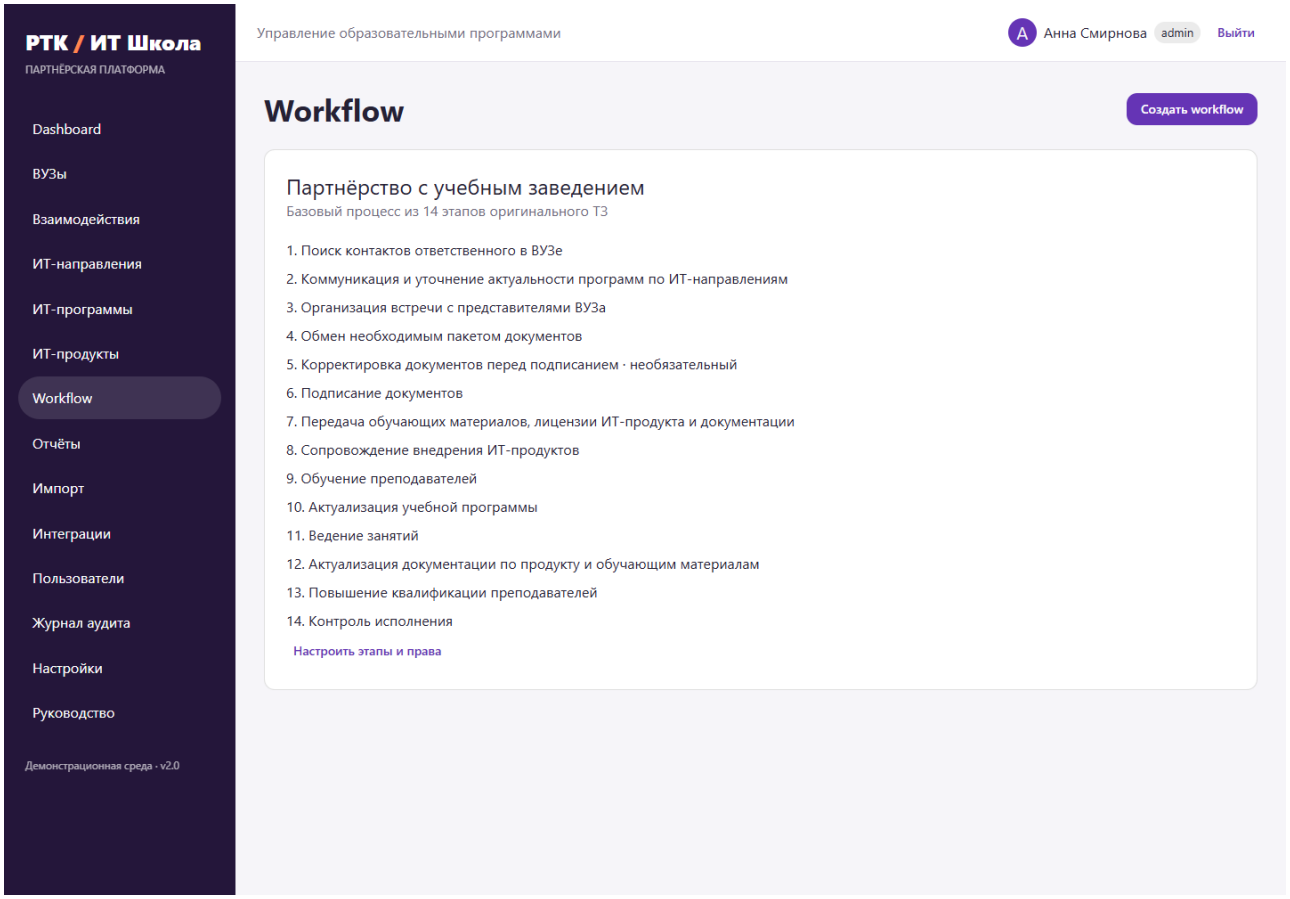
Для нового сотрудника создайте пользователя в realm Keycloak, установите пароль и realm role user/manager/admin. При первом входе запись Employee появится автоматически. Либо заранее внесите Keycloak ID в разделе «Пользователи». Затем в CRM задайте руководителя, локальную роль и активность. После первичной регистрации именно роль CRM определяет полномочия и проверяется в БД при каждом запросе; изменение realm role не переписывает существующую локальную роль. Для немедленного отзыва доступа выключите Employee в CRM и завершите сессии в Keycloak. Пароли меняются только в Keycloak.

USER — КАМ, менеджер по ВУЗам. MANAGER — руководитель КАМов. ADMIN — технический администратор. В seed пять КАМов с ролью user, один руководитель и один администратор. Руководитель видит непосредственно подчинённых, без рекурсивных иерархий. Назначение ВУЗа даёт доступ ко всем его взаимодействиям. Также сотрудник видит взаимодействия, где указан ответственным. Для полного снятия доступа снимите обе связи. Снятие назначения ВУЗа не удаляет исторические документы.



Справочники и workflow

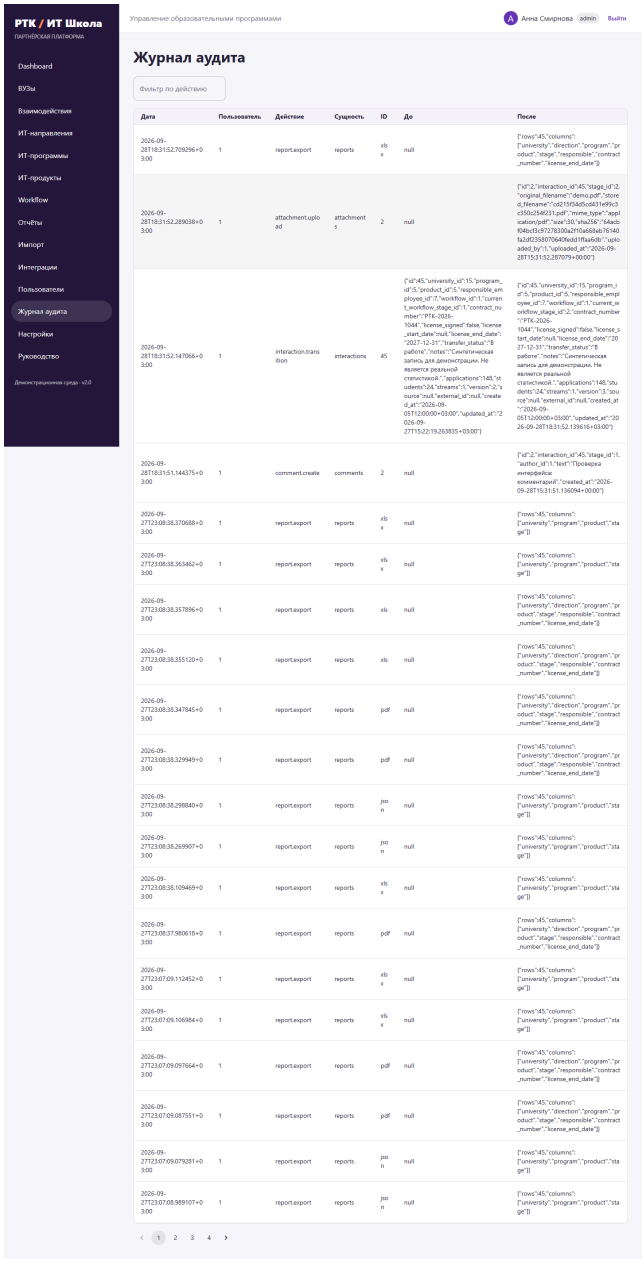
Разделы направлений, программ и продуктов поддерживают создание, редактирование и удаление. Удаление используемой записи блокируется внешними ключами. В редакторе workflow изменяются название, описание, порядок, активность, обязательность и разрешённые роли каждого этапа. Исторические этапы отключаются вместо физического удаления. Новые workflow создаются с произвольным количеством этапов, максимум 50. Изменение использующегося шаблона отражается во всех связанных взаимодействиях, история переходов сохраняется.



Конструктор workflow

Аудит и настройки

Аудит фиксирует создание/изменение/удаление сущностей, назначения, роли, workflow, переходы, импорт, синхронизацию и выгрузку отчётов. Записи содержат автора, время, тип/ID сущности, old_value и new_value. API не предоставляет удаления аудита. Настройка срока хранения носит регламентный характер и сама не удаляет данные. Размер файла задаёт MAX_UPLOAD_MB, адреса и токены интеграций — переменные окружения.



Журнал аудита

Интеграции

INTEGRATION_MODE=mock используется по умолчанию. Для реальных сервисов задайте http, LMS_API_URL, WEBSITE_API_URL, LMS_API_TOKEN, WEBSITE_API_TOKEN и INTEGRATION_ALLOWED_HOSTS. Разрешены только HTTPS и явно разрешённые имена узлов; редиректы запрещены. Контракт — массив нормализованных объектов, описанный в API.md. Внешние секреты не попадают в исходный код. Если контракт заказчика отличается, замените LMSClient или WebsiteClient, сохранив бизнес-сервис импорта.

Резервное копирование и восстановление

Перед обновлением сделайте логический дамп PostgreSQL и копию volume uploads. Для дампа используйте docker compose exec -T postgres pg_dump -U crm -d crm -Fc -f /tmp/crm.dump, затем docker compose cp postgres:/tmp/crm.dump ./crm.dump. Так сохраняются и CRM, и схема keycloak. Отдельно храните realm JSON, .env и файлы загрузок в защищённом месте. Redis содержит только кэш и может быть пересоздан. Проверяйте восстановление на отдельном стенде. Не выполняйте down -v, если данные нужно сохранить.

Ограничения

Compose — учебный стенд с start-dev, открытым HTTP localhost и известными демопаролями. Перед production необходимы действия из SECURITY.md и DEPLOYMENT.md. Docker-сборку в исходной среде нельзя считать проверенной: установлен portable Compose CLI, но отсутствует Docker Engine. Фактические результаты проверок приведены в ACCEPTANCE.md.

Архитектура

Компоненты и обработка данных

React SPA использует Router, TanStack Query, Axios, MUI и Recharts. JWT находится в памяти keycloak-js. Данные загружаются REST-запросами; изменения инвалидируют query cache без перезагрузки страницы. Фильтры, сортировка и пагинация выполняются сервером. Ограничение вариантов выбора в выпадающих списках — первая тысяча записей; большие каталоги требуют серверного autocomplete.

FastAPI API → services → repositories/SQLAlchemy → PostgreSQL. Pydantic проверяет входные и выходные DTO. SQL scope применяется до агрегации, формирования отчётов и выдачи файлов. JOIN-запросы и пакетное обогащение карточек предотвращают N+1 на списках. Версия Interaction и Workflow обеспечивает optimistic concurrency. Alembic — единственный механизм создания и изменения схемы.

Redis кэширует каталоги и агрегаты. Ключ включает ревизию, фильтры и текущую область доступа; изменение данных увеличивает ревизию. Старые значения истекают по TTL, обычно 60 секунд. Сбой Redis приводит к вычислению из PostgreSQL, а не к отказу основного функционала. Права и активность пользователя читаются непосредственно из БД.

Импорт проверяет XLS/XLSX и маппинг, преобразует типы, обрабатывает строки в savepoint. Повторный внешний импорт использует уникальный (source, external_id). Отчёты используют ту же выборку и права, что таблицы. XLSX создаёт openpyxl, XLS — xlwt, PDF — ReportLab со встроенным кириллическим шрифтом; Matplotlib экспортирует диаграммы. Архитектура синхронных worker threads поддерживает несколько отчётов одновременно; для объёмов свыше установленного лимита требуется отдельная очередь задач и object storage.

```
flowchart LR
Browser[React TypeScript SPA] -->|OIDC Code + PKCE| Keycloak
Browser -->|JWT REST| API[FastAPI]
API --> Services[Сервисы и RBAC]
Services --> ORM[SQLAlchemy repositories]
ORM --> PG[(PostgreSQL)]
Services --> Redis[(Redis cache)]
Services --> Storage[FileStorage: LocalVolume]
Services --> Reports[ReportLab / openpyxl / xlwt]
Services --> Adapters[LMSClient / WebsiteClient]
Adapters --> Sources[Mock или HTTPS API]
Services --> AI[AIWorkflowAssistant rules]
```

Развёртывание

```
flowchart TB
User[Браузер] -->|3000| Nginx[frontend: Nginx]
User -->|8080| KC[keycloak]
Nginx -->|api и handbook| Backend[backend: Uvicorn 8000]
User -->|8000 docs| Backend
Backend --> PG[postgres:5432]
KC -->|схема keycloak| PG
```

```
Backend --> Redis[redis:6379]
Backend --> Files[(uploads volume)]
PG --> Data[(postgres_data volume)]
Redis --> Cache[(redis_data volume)]
```

PostgreSQL и Redis не публикуют порты из Compose. Frontend запрашивает API через same-origin Nginx. Issuer Keycloak использует внешний localhost, JWKS доступен backend по внутреннему DNS. В native-проверке использовались отдельные процессы PostgreSQL/Keycloak/Uvicorn/Vite; она не заменяет сборку Linux-контейнеров.

Базовый процесс

```
flowchart TD
A[1 Контакты] --> B[2 Уточнение актуальности]
B --> C[3 Встреча]
C --> D[4 Обмен документами]
D --> E[5 Корректировка: опционально]
D --> F[6 Подписание]
E --> F
F --> G[7 Передача лицензий и материалов]
G --> H[8 Сопровождение внедрения]
H --> I[9 Обучение преподавателей]
I --> J[10 Актуализация программы]
J --> K[11 Занятия]
K --> L[12 Актуализация документации]
L --> M[13 Повышение квалификации]
M --> N[14 Контроль исполнения]
```

Эта схема — seed, а не ограничения кода. Администратор изменяет шаблоны, порядок, роли и активность. Перемещение требует текущую version и оставляет WorkflowTransition.

Сущности

```
erDiagram
University ||--o{ UniversityAssignment : assigned
Employee ||--o{ UniversityAssignment : responsible
University ||--o{ UniversityContact : contacts
University ||--o{ Interaction : partnerships
ITDirection ||--o{ ITProgram : programs
ITProgram ||--o{ Interaction : teaching
ITProduct ||--o{ Interaction : software
Employee ||--o{ Interaction : owner
Workflow ||--|{ WorkflowStage : stages
Workflow ||--o{ Interaction : process
Interaction ||--o{ WorkflowTransition : history
```

```
Interaction ||--o{ Comment : comments
```

```
Interaction ||--o{ Attachment : files
```

```
Employee ||--o{ AuditLog : audit
```

Перенос в Archi / ArchiMate

Откройте architecture.archimate в Archi. Бизнес-слой: роли KAM, руководитель, администратор; процесс «Сотрудничество с учебным заведением» и сервис «Контроль образовательных программ». Прикладной слой: SPA, CRM API, Keycloak, Reporting, Integration adapters. Объекты данных: University, Interaction, Workflow, Employee, AuditLog, Attachment. Технологический слой: браузер, Nginx, Python, PostgreSQL, Redis, volume и Linux Docker host. Связи Serving отражают обслуживание пользователя, Flow — обмен HTTP/JSON, Access — чтение/запись данных. Модель предназначена для дальнейшего уточнения инфраструктуры заказчика, доверенных зон и организационных ролей.

Заменяемые интерфейсы

FileStorage.save/path/delete допускает замену локального адаптера на S3; при переходе на presigned URLs нужно сохранить проверку доступа в API. LMSClient/WebsiteClient.fetch_records нормализуют контракт до импорта. AIWorkflowAssistant.suggest возвращает рекомендации без изменения состояния. Реальный LLM-провайдер должен отдельно согласовать передачу персональных данных.

API

Base URL: http://localhost:8000/api/v1. Интерактивное описание: /docs, OpenAPI: /openapi.json; снимок схемы приложен как openapi.json. Все защищённые запросы требуют Authorization: Bearer <access_token>. RS256, issuer и audience проверяются по JWKS Keycloak. API не принимает пароль.

Группы методов

| Методы | Путь | Назначение |

|---|---|---|

| GET | /auth/config, /auth/me | OIDC конфигурация, текущий сотрудник |

| GET, POST | /universities | Список и создание |

| GET, PUT, DELETE | /universities/{id} | Карточка, изменение, удаление |

| PUT | /universities/{id}/assignments | Назначения сотрудников |

| GET, POST | /universities/{id}/contacts | Контакты |

| PUT, DELETE | /universities/{id}/contacts/{cid} | Изменение контакта |

| GET, POST | /directions, /programs, /products | Справочники |

| GET, PUT, DELETE | /directions/{id}, /programs/{id}, /products/{id} | Запись справочника |

| GET, POST | /users | Сотрудники / регистрация |

| PUT | /users/{id} | Роль, руководитель, блокировка |

| GET, POST | /workflows | Шаблоны процессов |

| PUT | /workflows/{id} | Шаблон и этапы |

| GET, POST | /interactions | Взаимодействия |

| GET, PUT, DELETE | /interactions/{id} | Карточка, правка с version, удаление |

| GET | /interactions/{id}/allowed-stages | Разрешённые переходы |

| POST | /interactions/{id}/transition | Переход: to_stage_id, version, comment |

| GET | /interactions/{id}/timeline | История |

| GET, POST | /interactions/{id}/comments | Комментарии этапа |

| GET, POST | /interactions/{id}/attachments | Файлы: multipart file, stage_id |

| GET | /attachments/{id}/download | Защищённая загрузка |

| GET | /interactions/{id}/advice | Рекомендации |

| GET | /dashboard | Агрегаты и пять графиков |

| GET | /dashboard/chart/{png или pdf} | Диаграмма рейтинга |

| POST | /reports/preview, /reports/export | Отчёт с фильтрами и columns |

| GET | /import/template | XLSX шаблон |

| POST | /import/preview, /import/commit | XLS/XLSX анализ и импорт |

| GET | /integrations | Статусы адаптеров |

| GET | /integrations/{lms или website}/mock-data | Тестовый JSON |

| POST | /integrations/{lms или website}/sync | Синхронизация |

| GET | /audit | Журнал администратора |

| GET, PUT | /settings | Настройки |

Системный /health возвращает состояние БД, кэша и режим auth; без БД — 503, без Redis — degraded с работой без кэша. Документация доступна через /handbook/.

Форматы и фильтры

Пагинация: page от 1, page_size от 1 до 1000. Ответ {items,total,page,page_size}. Общие фильтры: start/end (YYYY-MM-DD), university_id, direction_id, program_id, product_id, responsible_employee_id, stage_id, q. Отчёты принимают те же поля в JSON, columns (ключи колонок) и format xlsx/xls/pdf/json. Даты возвращаются ISO, временные метки — UTC.

Единая ошибка: {"code":"VERSION_CONFLICT","message":"Запись изменена другим пользователем","details":{}}. Валидация добавляет details.fields с именем и причиной. Недоступные сущности возвращают 404, чтобы не раскрывать их существование. Все SQL-запросы параметризуются SQLAlchemy.

Контракт адаптеров

[{"external_id":"lms-1001","university":"Университет ИТМО","direction":"Разработка","program":"Основы Python","product":"РТК Облако","vendor":"Ростелеком","contract_number":"LMS-1001","license_signed":true,"license_end_date":"2027-12-31","transfer_status":"Передано","applications":120,"students":85,"streams":3,"notes":"Демонстрация"}]

Не более 10 000 записей и 20 МБ HTTP-ответа. Обязателен external_id; source задаёт маршрут. Каждая строка импортируется с правами инициатора. Параметр workflow_id выбирает шаблон для новых записей. Для существующего взаимодействия workflow не меняется. Итог {created,updated,skipped,errors:[{row,message}]} сохраняется в аудите и статусе интеграции.

Защита информации

Учебный проект реализует технические меры защиты, но не является автоматически сертифицированной системой и не подтверждает окончательное соответствие 152-ФЗ, 149-ФЗ или приказу ФСТЭК №117. Применимость требований, категория системы, модель угроз и полнота мер определяются заказчиком с профильными специалистами. Ниже описаны фактические свойства реализации и границы ответственности.

Реализовано

- Keycloak OIDC Authorization Code + PKCE S256; backend проверяет RS256, подпись, issuer, audience, exp, iat и sub. Direct password grant frontend отключён. Access token хранится только в памяти SPA.

- RBAC и серверная область доступа к ВУЗам/взаимодействиям. Проверяются отчёты, импорт, комментарии, файлы и интеграции. Роль и блокировка сотрудника читаются в БД без Redis, поэтому изменения применяются к следующему запросу.

- Валидация Pydantic, ограничения внешних ключей и уникальности; параметризованные ORM-запросы; optimistic concurrency.

- Файлы вне БД, случайные имена, проверка расширения/MIME/сигнатур/размера, защита от path traversal. Для Office ZIP проверяется структура и суммарный размер распаковки. Содержимое не исполняется, защищённая выдача — как attachment. Антивирусная проверка не реализована.

- Аудит before/after критичных операций. Нет API удаления аудита. Параметры входа и токены не записываются в бизнес-аудит. Аудит может содержать персональные данные изменяемых объектов и требует собственной политики хранения.

- CORS allowlist, X-Content-Type-Options, X-Frame-Options, Referrer-Policy, Permissions-Policy; HSTS для HTTPS backend. Excel-формулы экранируются при экспорте. HTML в PDF экранируется.

- HTTPS/allowlist/no redirects для внешних API. Секреты внешних систем — только env. PostgreSQL и Redis внутри Compose-сети. Приложение backend работает непривилегированным пользователем.

До промышленной эксплуатации

Сменить все опубликованные демонстрационные пароли и bootstrap-данные. Удалить демопользователей/фикстуры, отключить SEED_DEMO. Перевести Keycloak из start-dev в production, задать TLS, точные внешние адреса, redirect URI и CORS без wildcard доменов. Ограничить сетевой доступ, отключить публичные служебные порты, защитить БД отдельными учётными записями и секрет-хранилищем.

Заказчик обеспечивает правовые основания и цели обработки, минимизацию и сроки хранения ПДн, локальные акты, матрицу допуска, обучение сотрудников, резервное копирование с проверкой восстановления, шифрование носителей/каналов по своей модели угроз, управление ключами, мониторинг, реагирование на инциденты и актуализацию компонентов. Нужны внешний rate limiting/WAF, защита секретов, антивирус файлов, контроль уязвимостей и регулярное тестирование доступа. Изоляция и неизменяемость аудита на уровне инфраструктуры требуют отдельного решения; администратор БД технически способен изменить записи.

Демонстрационный Compose публикует HTTP localhost и использует общую БД с отдельной схемой Keycloak. Это упрощение учебного стенда. Производственная схема должна разделять роли БД и доверенные зоны. Итоговая оценка соответствия проводится после выбора инфраструктуры и организационных мер, а не на основании наличия Dockerfile или JWT.

Развёртывание и сопровождение

Docker / Linux

Требуются Docker Engine, Compose v2, доступ к интернету для первой сборки, ориентировочно 4 CPU, 8 ГБ RAM и 8 ГБ диска. Рекомендации по ресурсам — стартовая конфигурация, а не подтверждённая граница мощности.

```
cp .env.example .env
```

```
docker compose config
```

```
docker compose up --build -d
```

```
docker compose ps
```

```
docker compose logs -f backend
```

Windows PowerShell: Copy-Item .env.example .env. Используйте Docker Desktop с Linux containers. Адреса: frontend localhost:3000, backend localhost:8000, Swagger localhost:8000/docs, Keycloak localhost:8080. Остановить без потери данных: docker compose down.

На Windows можно запустить start.bat или powershell -File scripts/verify-compose.ps1: сценарий проверит Engine и Compose config, соберёт сервисы, дождётся healthchecks, проверит реальный Redis PING и Alembic. Он не устанавливает Docker/WSL и не меняет компоненты Windows. Перед Compose остановите локальные native-сервисы, занимающие те же порты.

FRONTEND_PORT, BACKEND_PORT, KEYCLOAK_PORT меняют опубликованные порты. При смене frontend-порта также измените redirectUris и webOrigins клиента crm-frontend в docker/keycloak-realm.json до первого запуска, либо в уже работающем realm через консоль Keycloak. Realm import не перезаписывает существующий realm при перезапуске. При смене hostname согласуйте KC_HOSTNAME, KEYCLOAK_ISSUER, redirect URI и CORS. Токен с issuer localhost не подходит для другого домена.

PostgreSQL init SQL создаёт схему keycloak только при инициализации пустого volume. Для старой БД выполните CREATE SCHEMA IF NOT EXISTS keycloak AUTHORIZATION crm;. Файлы живут в uploads, БД — postgres_data, Redis — redis_data. MAX_UPLOAD_MB ограничивает backend; если увеличиваете лимит выше 20 МБ, согласуйте client_max_body_size в docker/nginx.conf.

Для POSTGRES_PASSWORD в текущем Compose используйте случайную URL-safe строку (например, 32+ латинских букв и цифр): пароль включён в DATABASE_URL. Если нужны специальные символы, задайте отдельный URL с percent-encoding и исходный пароль PostgreSQL/Keycloak отдельно.

Миграции

```
docker compose exec backend alembic current
```

```
docker compose exec backend alembic upgrade head
```

```
docker compose exec backend alembic downgrade -1
```

Создавайте миграции в рабочей копии, где есть модели и доступ к БД:

```
cd backend
```

```
alembic revision --autogenerate -m "describe_change"
```

```
alembic upgrade head
```

```
alembic downgrade -1
```

Всегда просматривайте сгенерированный Python и проверяйте downgrade на отдельной БД. Первичный downgrade до base удаляет таблицы CRM: перед этим нужен backup. Изменять базу Keycloak миграциями CRM нельзя. Production startup не вызывает create_all.

Разработка без Docker

На текущем подготовленном Windows-стенде запустите start-local.bat. Он использует существующие .runtime/pgsql, .runtime/pgdata, .runtime/jdk-21*, .runtime/keycloak-26.2.5, .tools и node_modules; не скачивает программы и не меняет компоненты Windows. Все процессы запускаются скрыто, логи — .runtime/logs. Сценарий применяет миграции, сохраняет данные, проверяет issuer Keycloak и frontend API proxy. Он предназначен для этой демонстрационной БД; не направляйте его на production PostgreSQL. Redis необязателен для работы CRM: при отсутствии сервис продолжает работу без кэша.

Эти portable runtime не распространяются в репозитории. Для новой машины настройте сервисы по следующей инструкции или используйте Compose на Linux/Docker-хосте. Docker Compose здесь подготовлен, но интеграционный запуск контейнеров на данной Windows-машине не выполнялся из-за системного ограничения.

Поднимите PostgreSQL 17, Redis 7 и Keycloak 26.2.5, импортируйте realm. Python 3.12+ и Node 22.12+/24. Создайте virtualenv, установите backend/requirements-dev.txt. Из корня задайте PYTHONPATH=backend, DATABASE_URL и остальные переменные из .env.example; выполните Alembic и python -m app.db.seed, затем uvicorn app.main:app --host 127.0.0.1 --port 8000. npm ci и npm run dev запускают frontend на 3000. В Windows при блокировке npm.ps1 используйте npm.cmd.

Проверки

```
pytest backend/tests -q
ruff check backend/app backend/alembic backend/tests
ruff format --check backend/app backend/alembic backend/tests
npm test
npm run lint
npm run typecheck
npm run build
```

По умолчанию backend-тесты работают на изолированной SQLite БД. Для проверки PostgreSQL создайте ОТДЕЛЬНУЮ пустую БД и задайте TEST_DATABASE_URL. Тесты выполняют downgrade до base и уничтожают свои тестовые данные; никогда не передавайте рабочую БД. node scripts/browser-smoke.mjs использует локальный Edge, настоящий Keycloak и запущенный стенд. node scripts/load-check.mjs выполняет 50 параллельных запросов и 10 отчётов; результаты записываются в docs.

Более строгий сценарий: python scripts/load_test.py --users 50 --rounds 5. Он создаёт 50 независимых OIDC/PKCE-сессий, делает 250 запросов и 10 отчётов, затем завершает сессии. LOAD_USERNAME/LOAD_PASSWORD переопределяют демоучётку; секреты остаются в памяти процесса. Результат — docs/load-sessions-results.json. Тест предназначен для демонстрационного/тестового стенда и создаёт записи аудита экспорта.

Масштабирование

В учебном стенде используется один Uvicorn worker, пул SQLAlchemy 20 соединений + 40 overflow, синхронные отчёты в thread pool. Размер выборки отчёта ограничен 50 000, импорта — 10 000 строк. Для тяжёлых производственных объёмов вынесите генерацию в очередь workers с ограничением конкурентности и сохранением результатов. Нагрузочный тест на демонстрационных данных не

является SLA для реальных объёмов. Следите за длительностью SQL, CPU, памятью, ожиданием пула и cache hit rate.

Фактическая приёмка v2

Дата проверки: 27 сентября 2026. Среда: Windows, Python 3.14.6, Node 24.20, PostgreSQL 17 portable, Keycloak 26.2.5 portable, Edge. Dockerfile рассчитан на Python 3.12/Linux; Linux-контейнеры в этой среде не запускались.

Выполненные проверки

| Проверка | Результат |

|---|---|

| Полное чтение PDF и финальная сверка | CHECKLIST.md и REQUIREMENTS-V2.md |

| Alembic upgrade → downgrade base → upgrade | Пройдено в тестовой PostgreSQL и SQLite |

| Seed / повторный seed | 15 ВУЗов, 5 направлений, 10 программ, 8 продуктов, 5 КАМов, 45 взаимодействий; без дублей |

| Backend pytest на PostgreSQL | 14 passed |

| Frontend Vitest / RTL | 5 passed |

| Ruff / ESLint / TypeScript | Пройдены |

| Vite production build | Собрана, JS разделён на chunks |

| Реальный Keycloak OIDC + PKCE | Вход admin, manager, user в Edge |

| Основные страницы / мобильное меню / 403 | Браузерные проверки и скриншоты |

| UI: комментарий / переход / файл / XLSX / Excel preview | Пройдены, полных перезагрузок 0 |

| CRUD / RBAC / scope / JWT claims / блокировка | Автотесты API |

| Отчёты XLSX / XLS / PDF / JSON | Автотесты содержимого/сигнатур |

| Excel XLS/XLSX, частичный откат ошибочной строки | Автотесты |

| LMS / Website idempotency | Повторная синхронизация не создаёт дубль |

| Redis invalidation / изоляция статистики | Автотесты с fakeredis |

| Compose config | Валиден, официальный standalone Compose CLI |

| Docker build | Не выполнен: Docker Engine отсутствует |

Замеры

Подробности: load-results.json, ui-performance.json, browser-results.json. Замеры сделаны на loopback с демонстрационными данными, без реального Redis. 50 запросов одновременно к спискам/аналитике: 0 ошибок, p50 1228 мс, p95 1340 мс, максимум 1352 мс. 10 одновременно формируемых отчётов разных форматов/колонок: 0 ошибок, p95 653 мс. Обычные действия UI: комментарий 221 мс, переход 480 мс, preview отчёта 320 мс.

Это проверка 50 конкурентных запросов с действующей сессией, а не длительная модель 50 независимых пользователей. Требование ≤1 секунды при такой нагрузке не подтверждено. Нужен отдельный прогон Linux Compose с Redis, целевыми объёмами и профилем пользователей. Скрипты оставлены для воспроизведения.

Дополнительный эксперимент с двумя native Windows Uvicorn workers ухудшил хвост задержек: p95 около 30 секунд, хотя ошибок HTTP не было. Результат сохранён в load-results-two-workers.json; эта конфигурация не принята. Итоговый Compose и локальный API оставлены с одним worker. Автоматически переносить результаты Windows multiprocessing на Linux нельзя.

Definition of Done

Дополнительный прогон после продолжения работ: scripts/load_test.py создаёт 50 отдельных сессий Keycloak через Authorization Code + PKCE, выполняет 5 синхронных раундов (250 запросов), затем 10 одновременных отчётов и завершает тестовые сессии. Токены не сохраняются. Результат в load-sessions-results.json: 0 ошибок, p50 479 мс, p95 1217 мс; отчёты p95 582 мс. До оптимизации медиана была 591 мс, но общий p95 нового прогона выше из-за первого раунда. Удалены лишние запросы к БД при недоступном кэше, три агрегата Dashboard объединены в один SQL round trip. Требование ≤1 с по-прежнему не объявляется закрытым.

Функциональные пункты 6–28 реализованы: авторизация/роли, Dashboard, CRUD ВУЗов/программ/продуктов, workflow/переходы, комментарии, файлы, Excel, фильтры, XLSX/PDF, Swagger, две mock-интеграции, seed, README, документация и тесты. Критических TODO/заглушек в бизнес-операциях нет. Интерфейсы Protocol и mock-адаптеры — предусмотренные точки расширения.

Пункты 1–5, относящиеся к docker compose up --build, нельзя отметить пройденными: конфигурация проверена, но Docker Engine недоступен. PostgreSQL, Keycloak, backend и frontend запущены и проверены отдельными локальными процессами. CI workflow подготовлен для сборки и smoke-проверки настоящего Compose на Ubuntu, но его внешнее выполнение здесь не заявляется.

Проверка без Docker, 29.09.2026

- start-local.bat / scripts/start-local.ps1: подтверждён запуск остановленных PostgreSQL, Keycloak, backend и frontend. Проверены миграции, issuer и API proxy; системные компоненты Windows не устанавливались.

- Backend: 14/14 тестов на SQLite и отдельно 14/14 на PostgreSQL crm_test; fixture проверяет upgrade → downgrade → upgrade. Ruff проходит.

- Браузер: admin, manager, user — вход успешен, JavaScript-ошибок нет; основные страницы и ограничения ролей проверены.

- Пользовательские операции: комментарий 196 мс, переход workflow 439 мс, preview отчёта 238 мс, загрузка файла, скачивание XLSX и Excel preview успешны. Перезагрузок страницы при редактировании — 0. Результаты в browser-results.json и ui-performance.json.

- docker compose config --quiet проходит. Сборка и интеграционный запуск контейнеров не выполнялись из-за системного ограничения Windows; Docker/WSL2 по указанию пользователя больше не устанавливаются.

- Health: PostgreSQL работает, авторизация Keycloak работает, Redis недоступен (degraded); работа CRM без кэша подтверждена.

История проверок и известные ограничения

Повторная проверка после перезагрузки Windows, 28.09.2026

Локальные PostgreSQL, Keycloak, backend и собранный frontend повторно запущены. Браузерная проверка admin/manager/user прошла без JavaScript-ошибок. Сценарий действий подтвердил комментарий (936 мс), переход workflow (165 мс), preview отчёта (211 мс), загрузку файла, XLSX download и Excel preview; полных перезагрузок при редактировании — 0. Health: database=ok, redis=degraded. Это проверка native-стенда, не Docker Compose. Состояние подготовки Windows

описано в WINDOWS-SETUP-STATUS.md.

- Реальные LMS/сайт не подключены без контрактов и доступов заказчика.

- Docker build/healthcheck в Linux и настоящий Redis остаются для проверки на Docker-хосте.

- Формирование отчётов синхронное, до 50 000 строк; импорт до 10 000. Для больших объёмов нужна очередь.

- Выборщики UI загружают до 1000 элементов; иерархия руководителя учитывает прямых подчинённых.

- Настройка retention не удаляет данные автоматически. Антивирусная обработка файлов и инфраструктурная неизменяемость аудита не реализованы.

- Production TLS, пароли, резервное копирование, мониторинг и итоговое нормативное соответствие требуют настройки заказчика.

- Публичный репозиторий и hosted prototype не опубликованы.

- В локальном TestClient есть предупреждение Starlette об устаревающем httpx transport; тесты проходят.